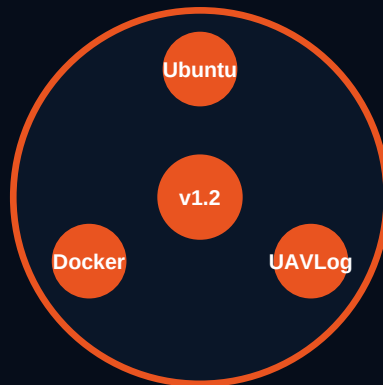




UAVLogBook

Ubuntu Docker Install Guide

Ubuntu versions	20.04 LTS · 22.04 LTS · 24.04 LTS
Architecture	x86_64 (amd64) and ARM64 both supported
Covers	Fresh server · Dev machine · VPS / cloud
Time to complete	~20 minutes from blank Ubuntu install

Version 1.2 · June 2025

Table of Contents

1	Prerequisites & System Check	3
2	Install Docker Engine	4
3	Install Docker Compose v2	5
4	Post-Install: User Groups & Auto-start	6
5	Download & Extract UAVLogBook	7
6	Configure .env	8
7	Start — Development Mode	10
8	Start — Production Mode (no SSL)	12
9	Start — Production with Nginx + SSL	13
10	First Login & Test	15
11	Firewall Setup (ufw)	16
12	Auto-start on Boot (systemd)	17
13	Useful Commands	18
14	Updating UAVLogBook	19
15	Backup & Restore	20
16	Troubleshooting	21

1 Prerequisites & System Check

These steps assume a fresh or existing Ubuntu installation. Run everything as your normal user — `sudo` will be used where root is needed. Do NOT run as root.

Supported Ubuntu versions

Version	Kernel	Status	Notes
Ubuntu 24.04 LTS	6.8	✓ Best	Noble Numbat — latest LTS, recommended
Ubuntu 22.04 LTS	5.15	✓ Great	Jammy Jellyfish — most common, well tested
Ubuntu 20.04 LTS	5.4	✓ Good	Focal Fossa — supported until 2025
Ubuntu 18.04 LTS	4.15	■ Old	EOL — upgrade before installing Docker

Minimum hardware

RAM	2 GB minimum	4 GB recommended (for comfortable dev use)
Disk	10 GB free	50 GB+ if storing large video files
CPU	1 core minimum	2+ cores for faster log parsing
Network	Internet access	Required for first-run image pull (~1.4 GB total)

Quick system check — run these first

Terminal

```
# Check Ubuntu version
$ lsb_release -a

# Check available disk space (need 10 GB+ free)
$ df -h /

# Check RAM
$ free -h

# Check internet connectivity
$ curl -s https://hub.docker.com > /dev/null && echo 'Internet OK'

# Update package lists before installing anything
$ sudo apt-get update
```

2 Install Docker Engine

Use the official Docker convenience script — the fastest and most reliable method for Ubuntu. It works on 20.04, 22.04, and 24.04.

1

Remove any old Docker versions

Prevents conflicts with outdated packages

Terminal

```
$ sudo apt-get remove -y docker docker-engine docker.io containerd runc
# It is OK if this says 'none of these packages are installed'
```

2

Run the official install script

Downloads and installs Docker Engine, CLI, containerd, and Compose plugin

Terminal

```
$ curl -fsSL https://get.docker.com -o get-docker.sh
$ sudo sh get-docker.sh

# This takes 1-3 minutes. You will see output like:
# + sh -c apt-get update -qq
# + sh -c DEBIAN_FRONTEND=noninteractive apt-get install ...
# ...
# Client: Docker Engine - Community
# Version: 26.x.x
```

✓ Expected output:

```
Client: Docker Engine - Community
Version:      26.x.x
API version:  1.45
Go version:   go1.21.x
Server: Docker Engine - Community
Engine:
Version:      26.x.x
API version:  1.45 (minimum version 1.24)
```

3

Verify Docker is installed

Terminal

```
$ sudo docker --version
```

✓ Expected output:

```
Docker version 26.x.x, build xxxxxxxx
```

Why use the convenience script?

The script automatically detects your Ubuntu version, adds the correct Docker APT repository with GPG verification, and installs the latest stable release. It is the method recommended by Docker for Linux servers. The script source is available at: <https://github.com/docker/docker-install>

3 Install Docker Compose v2

The Docker convenience script from Step 2 already installs the Compose plugin. Run this check to confirm, then install manually if needed.

1 Check if Compose v2 is already installed

Terminal

```
$ docker compose version
```

✓ *Expected output:*

```
Docker Compose version v2.x.x
```

If the above command works, skip to Section 4. If you get 'unknown command' or 'docker: compose is not a docker command', install the plugin manually:

2 Install Compose plugin manually (only if needed)

Terminal

```
$ sudo apt-get update
$ sudo apt-get install -y docker-compose-plugin

# Verify
$ docker compose version
```

✓ *Expected output:*

```
Docker Compose version v2.x.x
```

docker compose vs docker-compose

There are two versions: 'docker compose' (v2, space, plugin — what we use) and 'docker-compose' (v1, hyphen, separate Python binary — old). UAVLogBook requires v2. If you have only v1, run: `sudo apt-get remove docker-compose && sudo apt-get install docker-compose-plugin`

4 Post-Install: User Groups & Auto-start

1

Add your user to the docker group

So you can run docker without sudo every time

Terminal

```
# Replace 'yourname' with your actual username
$ sudo usermod -aG docker $USER

# Apply the group membership immediately (no logout needed)
$ newgrp docker

# Confirm it worked – should list 'docker'
$ groups
```

✓ Expected output:

```
yourname sudo docker ... ← docker should appear in the list
```

2

Enable Docker to start automatically on boot

Terminal

```
$ sudo systemctl enable docker
$ sudo systemctl start docker

# Check it is running
$ sudo systemctl status docker
```

✓ Expected output:

```
● docker.service - Docker Application Container Engine
   Loaded: loaded (/lib/systemd/system/docker.service; enabled)
   Active: active (running) since ...
     Docs: https://docs.docker.com
```

3

Run a test to confirm everything works

Terminal

```
# This should run WITHOUT sudo after the group change above
$ docker run hello-world
```

✓ *Expected output:*

```
Hello from Docker!
```

```
This message shows that your installation appears to be working correctly.
```

If 'docker run' still needs sudo

The newgrp docker command applies the group to the current session. If you open a new terminal and it still needs sudo, log out completely and log back in — the group membership will then be permanent.

5 Download & Extract UAVLogBook

Option A — Transfer from your computer with scp

If you downloaded UAVLogBook_v1.2.zip on another machine (Windows/macOS), copy it to Ubuntu with scp:

Your local computer — not Ubuntu

```
# Run this on your LOCAL computer, not the Ubuntu server
# Replace 192.168.1.100 with your Ubuntu machine's IP
$ scp UAVLogBook_v1.2.zip yourname@192.168.1.100:~/

# Windows PowerShell:
PS> scp UAVLogBook_v1.2.zip yourname@192.168.1.100:~/
```

Option B — Download directly on Ubuntu

If the zip is accessible via a URL (e.g. from a file host or your own server):

Terminal — Ubuntu

```
# On Ubuntu, download directly
$ cd ~
$ wget https://your-download-url/UAVLogBook_v1.2.zip
# or:
$ curl -L -o UAVLogBook_v1.2.zip https://your-download-url/UAVLogBook_v1.2.zip
```

Extract and enter the project folder

Terminal — Ubuntu

```
# Install unzip if not present
$ sudo apt-get install -y unzip

# Extract
$ cd ~
$ unzip UAVLogBook_v1.2.zip

# Enter the project directory
$ cd uavlogbook_v3

# Confirm the key files are present
$ ls
```

✓ *Expected output:*

docker-compose.yml	docker-compose.dev.yml	Makefile
.env.example	server/	web/
mobile/	docker/	README.md

Install make (for convenience commands)

Terminal — Ubuntu

```
$ sudo apt-get install -y make
```

```
# Verify
```

```
$ make --version
```

Project location tip

For a server install, consider placing the project under `/opt/uavlogbook/` instead of your home directory. Replace `~` with `/opt` in the commands above and use `sudo mkdir /opt/uavlogbook`. This is a common convention for services.

6 Configure .env

All passwords, secrets, and settings live in a single .env file. Create it from the template and edit it before starting Docker.

1 Create the .env file from the template

Terminal

```
$ cd ~/uavlogbook_v3
$ cp .env.example .env
```

2 Generate a secure JWT secret

Terminal

```
# Generate a 64-character random hex string
$ openssl rand -hex 32
```

✓ Expected output:

```
a8f3d2c1b4e5f6a7b8c9d0e1f2a3b4c5d6e7f8a9b0c1d2e3f4a5b6c7d8e9f0a1
```

Copy this output — you will paste it as JWT_SECRET in the next step.

3 Edit .env and set your values

Terminal

```
# Open with nano (simple) or vim
$ nano .env

# Find and change these lines:
MYSQL_ROOT_PASSWORD=choose_a_strong_root_password
MYSQL_PASSWORD=choose_a_strong_app_password
JWT_SECRET=paste_your_openssl_output_here

# Save and exit nano: press Ctrl+X, then Y, then Enter
```

4 Complete .env reference

All available settings explained

MYSQL_ROOT_PASSWORD

REQUIRED

MySQL root password. Any strong password. Not used directly by the app.

MYSQL_DATABASE

uavlogbook

Database name. Leave as-is.

MYSQL_USER	<code>uavlogbook_user</code>	App database username. Leave as-is.
MYSQL_PASSWORD	REQUIRED	App database password. Must be different from root password.
JWT_SECRET	REQUIRED — 64 chars	Token signing key. Generate with: <code>openssl rand -hex 32</code>
ANTHROPIC_API_KEY	(blank = disabled)	Claude AI key for smart log format detection. Leave blank — AI is optional.
VITE_API_URL	<code>/api/v1</code>	API path baked into the React build. Leave as <code>/api/v1</code> in Docker.
ALLOWED_ORIGINS	<code>http://localhost,...</code>	CORS origins. Add your server IP or domain, e.g. <code>http://192.168.1.50:8080</code>
NGINX_SERVER_NAME	<code>yourdomain.com</code>	Your domain. Only used with the Nginx production profile and SSL.
PHP_MEMORY_LIMIT	<code>256M</code>	PHP memory per request. 256M is fine. Raise to 512M for very large log files.
PHP_UPLOAD_MAX_FILESIZE	<code>8192M</code>	Maximum file size. 8192M = 8 GB, covering the largest FPV video files.
APP_PORT	<code>8080</code>	Port the app binds on the host. Change if 8080 is in use.

5 Lock down the .env file permissions

Terminal

```
# Make .env readable only by your user — keeps passwords safe
$ chmod 600 .env

# Verify
$ ls -la .env
```

✓ *Expected output:*

```
-rw----- 1 yourname yourname 854 Jun  1 14:22 .env
```

7 Start — Development Mode

Development mode mounts your source files live into the containers. Edit PHP or React files and changes are reflected immediately — no container rebuild required.

1 Start the dev stack

Terminal

```
$ cd ~/uavlogbook_v3

# Using make (recommended):
$ make dev

# Without make:
$ docker compose -f docker-compose.yml -f docker-compose.dev.yml up --build
```

First run takes longer

On first start Docker pulls ~1.4 GB of images and builds the PHP container (compiles extensions like pdo_mysql, gd, zip). This takes 5–15 minutes depending on your internet speed and CPU. Every subsequent start is under 30 seconds.

2 Watch the startup logs — what to expect

Expected terminal output

```
uavlogbook_db      | Starting MySQL...
uavlogbook_db      | ready for connections.      ← MySQL is ready

uavlogbook_app     | Building image...           ← Only on first run
uavlogbook_app     | Successfully built xxxxxxxx
uavlogbook_app     | AH00094: Command line: ...   ← Apache started

uavlogbook_webdev  | VITE v5.x ready in 1234 ms   ← React dev server ready
uavlogbook_webdev  | → Local: http://localhost:3000/
```

3 Open the app in your browser

Browser URLs

```
# If running on the SAME machine as the browser:
http://localhost:3000      ← React app with hot reload (Vite HMR)
http://localhost:8080      ← App via Apache (production-like)
http://localhost:8080/api/v1 ← API direct

# If accessing from ANOTHER machine on the network:
# Replace localhost with your Ubuntu machine's IP address
# Find your IP with:
$ ip addr show | grep 'inet ' | grep -v 127
# Example: http://192.168.1.50:3000
```

4 Verify the API is responding

Terminal

```
$ curl -s -X POST http://localhost:8080/api/v1/auth/login \
-H 'Content-Type: application/json' \
-d '{"email":"x","password":"x"}'
```

✓ *Expected output:*

```
{"error":"Invalid credentials"} ← API is working correctly
```

Dev mode features

PHP hot-edit :8080 — Edit files in server/ — refresh browser to see changes immediately. No container restart needed. The directory is bind-mounted read-write.

React HMR :3000 — Edit web/src/ files — browser updates in milliseconds without page refresh. Powered by Vite Hot Module Replacement.

MySQL on :3306 — Connect any GUI tool (DBeaver, MySQL Workbench, TablePlus) to localhost:3306 with the credentials from .env.

Xdebug on :9003 — PHP step-debugger. Attach VS Code or PhpStorm to localhost:9003 — see Section 13 for VS Code launch.json config.

PHP errors visible — Errors appear in the browser response body (PHP_DISPLAY_ERRORS=On). In production mode this is disabled.

5 Stop the dev stack

Terminal

In the terminal where it's running: press Ctrl+C

Or from another terminal:

\$ **make down**

or:

\$ **docker compose down**

Database and uploads are preserved in Docker volumes

Start again any time with: make dev

8 Start — Production Mode (no SSL)

Use this for a local server, home lab, or when you already have a reverse proxy (Nginx, Caddy, Traefik, Cloudflare) in front. The React app is pre-built, Opcache is on, MySQL is not exposed.

Terminal

```
$ cd ~/uavlogbook_v3
$ make up
# or:
$ docker compose up -d

# Check all containers started:
$ make ps
# or:
$ docker compose ps
```

✓ Expected output:

NAME	IMAGE	STATUS	PORTS
uavlogbook_db	mysql:8.0	Up (healthy)	3306/tcp
uavlogbook_builder	node:20-alpine	Exited (0)	← Normal — exits after build
uavlogbook_app	uavlogbook-app	Up (healthy)	0.0.0.0:8080->80/tcp

Access URLs

```
# Open the app:
http://localhost:8080

# Or from another machine:
http://192.168.1.50:8080 ← use your Ubuntu IP

# Follow logs to confirm all is well:
$ make logs
# or:
$ docker compose logs -f
```

builder container exits with code 0 — this is correct

The builder container compiles the React app once, then exits. Status 'Exited (0)' means it completed successfully. It does NOT restart — it only runs again if you manually rebuild with: `make rebuild-web`

9 Start — Production with Nginx + SSL

Use this for a real internet-facing server. Nginx adds SSL termination, HTTP→HTTPS redirect, rate limiting, and caching. A free Let's Encrypt certificate is provisioned automatically.

DNS must point to this server

Before running certbot, your domain's A record must resolve to this server's public IP address. Check propagation: `nslookup yourdomain.com` — it should return your server's IP. DNS can take up to 60 minutes to propagate.

1 Open firewall ports 80 and 443

Terminal

```
$ sudo ufw allow 80/tcp
$ sudo ufw allow 443/tcp
$ sudo ufw reload
$ sudo ufw status
```

✓ Expected output:

Status: active

To	Action	From	
80/tcp	ALLOW	Anywhere	
443/tcp	ALLOW	Anywhere	
22/tcp	ALLOW	Anywhere	← SSH (should already be open)

2 Edit .env with your domain

Terminal

```
$ nano .env

# Set your domain:
NGINX_SERVER_NAME=yourdomain.com
ALLOWED_ORIGINS=https://yourdomain.com
VITE_API_URL=/api/v1
```

3 Start full production stack with Nginx

Terminal

```
$ make prod
# or:
$ docker compose --profile production up -d

# Check Nginx started:
$ docker compose ps
```

4**Get a free SSL certificate from Let's Encrypt****Terminal**

```
$ make ssl D=yourdomain.com E=admin@yourdomain.com
# or:
$ bash docker/nginx/certbot.sh yourdomain.com admin@yourdomain.com

# What this does:
# 1. Runs certbot in a temporary container
# 2. Proves domain ownership via /.well-known/acme-challenge/
# 3. Copies fullchain.pem + privkey.pem into the ssl_certs volume
# 4. Reloads Nginx to use the new certificate

# On success:
# SSL configured for yourdomain.com
```

✓ *Expected output:*

```
Congratulations! Your certificate and chain have been saved.
SSL configured for yourdomain.com
```

5**Verify HTTPS is working****Terminal**

```
# Test HTTPS redirect
$ curl -I http://yourdomain.com
# Expected: HTTP/1.1 301 Moved Permanently ← redirects to HTTPS

# Test HTTPS
$ curl -I https://yourdomain.com
# Expected: HTTP/2 200

# Or open in browser:
https://yourdomain.com
```

6

Set up automatic SSL renewal

Let's Encrypt certificates expire after 90 days. Add a cron job to renew automatically:

Terminal — crontab

```
$ crontab -e
```

```
# Add this line (runs at 3:00 AM on the 1st of every other month):
```

```
0 3 1 */2 * cd /home/yourname/uavlogbook_v3 && bash docker/nginx/certbot.sh yourdomain.com admin@yourdomain.com
```

10 First Login & Test

1 Open the web app

Development: http://localhost:3000
Production: http://localhost:8080
Production SSL: https://yourdomain.com

2 API health check

Terminal

```
$ curl -s -X POST http://localhost:8080/api/v1/auth/login \  
-H 'Content-Type: application/json' \  
-d '{"email":"x","password":"x"}'
```

✓ Expected output:

```
{"error":"Invalid credentials"}
```

← This response means: Apache ✓ PHP ✓ MySQL ✓ API ✓

3 Register your first account

- Click Register on the login screen
- Enter email, password (min 8 chars), and display name
- Click Create Account
- The first account registered is automatically admin

4 Import a flight log

- Click Import Log in the left sidebar
- Drag and drop a flight log file onto the drop zone
- Supported formats: .bin .tlog .ulg .skylog .csv .gpx .kml .bbl
- The AI engine analyses the file and detects the format automatically
- Open the flight to view the analysis dashboard with all telemetry charts

11 Firewall Setup (ufw)

Ubuntu's built-in firewall (ufw) should be configured to allow only the ports you actually need. This is especially important for internet-facing servers.

Check firewall status

Terminal

```
$ sudo ufw status verbose
```

Recommended rules

Terminal

```
# Allow SSH first – CRITICAL: do this before enabling ufw
$ sudo ufw allow ssh

# or for non-standard SSH port, e.g. 2222:
$ sudo ufw allow 2222/tcp

# For production (Nginx SSL):
$ sudo ufw allow 80/tcp
$ sudo ufw allow 443/tcp

# For development only (allow app port from local network):
# Replace 192.168.1.0/24 with your local subnet
$ sudo ufw allow from 192.168.1.0/24 to any port 8080

# Enable the firewall
$ sudo ufw enable

# Verify
$ sudo ufw status numbered
```

✓ Expected output:

Status: active

	To	Action	From
[1]	22/tcp	ALLOW IN	Anywhere
[2]	80/tcp	ALLOW IN	Anywhere
[3]	443/tcp	ALLOW IN	Anywhere

Never block port 22 before enabling ufw

If you enable ufw without allowing SSH first, you will lock yourself out of the server. Always run: `sudo ufw allow ssh` BEFORE `sudo ufw enable`. If you are on a cloud provider (AWS, DigitalOcean, etc.) check their firewall/security group settings as well — they operate independently of ufw.

12 Auto-start on Boot (systemd)

Create a systemd service so UAVLogBook starts automatically when the server boots, and restarts automatically if it crashes.

1 Create the systemd service file

Terminal

```
$ sudo nano /etc/systemd/system/uavlogbook.service
```

/etc/systemd/system/uavlogbook.service — paste this content

```
[Unit]
Description=UAVLogBook Docker Stack
Requires=docker.service
After=docker.service network-online.target

[Service]
Type=oneshot
RemainAfterExit=yes
WorkingDirectory=/home/yourname/uavlogbook_v3
ExecStart=/usr/bin/docker compose up -d
ExecStop=/usr/bin/docker compose down
TimeoutStartSec=300

[Install]
WantedBy=multi-user.target
```

Replace /home/yourname/uavlogbook_v3 with your actual project path.

2 Enable and start the service

Terminal

```
# Reload systemd to pick up the new service
$ sudo systemctl daemon-reload

# Enable – starts automatically on every boot
$ sudo systemctl enable uavlogbook

# Start it now
$ sudo systemctl start uavlogbook

# Check status
$ sudo systemctl status uavlogbook
```

✓ *Expected output:*

- uavlogbook.service - UAVLogBook Docker Stack
Loaded: loaded (/etc/systemd/system/uavlogbook.service; enabled)
Active: active (exited) since ...

Terminal

Test by rebooting

\$ **sudo reboot**

After reboot – check containers are running

\$ **cd ~/uavlogbook_v3 && docker compose ps**

13 Useful Commands

All make commands run from the `uavlogbook_v3/` directory. The full docker compose equivalent is shown for reference.

Starting & stopping

<code>make dev</code>	Dev mode — foreground, Ctrl+C to stop
■ <code>docker compose -f docker-compose.yml -f docker-compose.dev.yml up --build</code>	
<code>make dev-d</code>	Dev mode — background (silent)
■ <code>docker compose -f docker-compose.yml -f docker-compose.dev.yml up -d --build</code>	
<code>make up</code>	Production mode — background, no Nginx
■ <code>docker compose up -d</code>	
<code>make prod</code>	Production + Nginx on ports 80/443
■ <code>docker compose --profile production up -d</code>	
<code>make down</code>	Stop all containers, keep data
■ <code>docker compose --profile production down</code>	
<code>make ps</code>	Show container status
■ <code>docker compose ps</code>	

Logs

<code>make logs</code>	Stream all container logs (Ctrl+C to stop)
■ <code>docker compose logs -f</code>	
<code>make logs-app</code>	PHP / Apache logs only
■ <code>docker compose logs -f app</code>	
<code>make logs-db</code>	MySQL logs only
■ <code>docker compose logs -f db</code>	
<code>docker compose logs --tail=100 app</code>	Last 100 lines of app logs

Shells & debugging

<code>make shell</code>	Bash shell inside PHP container
■ <code>docker compose exec app bash</code>	
<code>make db-shell</code>	MySQL interactive shell
■ <code>docker compose exec db mysql -u... -p...</code>	
<code>docker stats</code>	Real-time CPU/RAM usage per container
<code>docker compose exec app php -m</code>	List PHP extensions loaded in container

Maintenance

<code>make rebuild-web</code>	Force React app rebuild (after editing web/src/ in production)
■ Remove web_dist volume + re-run builder	
<code>make rebuild-app</code>	Rebuild PHP Docker image (after Dockerfile changes)

■ docker compose build --no-cache app

make backup

Backup database to ./backups/

■ mysqldump → backups/uavlogbook_TIMESTAMP.sql

make restore F=backups/file.sql

Restore database from a SQL backup file

■ mysql < file

make ssl D=domain E=email

Get or renew Let's Encrypt SSL certificate

■ bash docker/nginx/certbot.sh

make clean

■ DESTRUCTIVE — removes containers + all volumes (data lost)

■ docker compose down -v --remove-orphans

14 Updating UAVLogBook

Update PHP server code

Terminal

```
# Copy new server/ files to the project directory
$ cp -r /path/to/new/server/* ~/uavlogbook_v3/server/

# In dev mode – changes are live immediately, no restart needed

# In production mode – restart the app container:
$ docker compose restart app
```

Update the React web app

In production mode the React app is pre-built. After changing web/src/ files, rebuild:

Terminal

```
$ make rebuild-web
# or manually:
$ docker compose stop builder
$ docker volume rm uavlogbook_web_dist
$ docker compose up builder
$ docker compose restart app
```

Update Docker base images (security patches)

Terminal

```
# Pull latest versions of all base images
$ docker compose pull

# Rebuild the custom PHP image with the updated base
$ docker compose build --no-cache app

# Restart everything
$ docker compose up -d

# Remove old unused images to free disk space
$ docker image prune -f
```

15 Backup & Restore

Database backup

Terminal

```
$ cd ~/uavlogbook_v3

# Quick backup using make:
$ make backup
# Saves to: backups/uavlogbook_YYYYMMDD_HHMMSS.sql

# Manual backup:
$ mkdir -p backups
$ source .env
$ docker compose exec -T db mysqldump \
  -u$MYSQL_USER -p$MYSQL_PASSWORD $MYSQL_DATABASE \
  > backups/uavlogbook_$(date +%Y%m%d_%H%M%S).sql

# Check backup size
$ ls -lh backups/
```

Database restore

Terminal

```
# Restore from a backup file:
$ make restore F=backups/uavlogbook_20250601_140000.sql

# Manual restore:
$ source .env
$ docker compose exec -T db mysql \
  -u$MYSQL_USER -p$MYSQL_PASSWORD $MYSQL_DATABASE \
  < backups/uavlogbook_20250601_140000.sql
```

Backup uploaded files (logs + videos)

Terminal

```
# Logs and videos are in Docker named volumes.
# Copy them out using a temporary alpine container:

$ docker run --rm \
  -v uavlogbook_uploads_logs:/data \
  -v ~/backups:/backup \
  alpine tar czf /backup/logs_$(date +%Y%m%d).tar.gz -C /data .

$ docker run --rm \
  -v uavlogbook_uploads_videos:/data \
  -v ~/backups:/backup \
  alpine tar czf /backup/videos_$(date +%Y%m%d).tar.gz -C /data .
```

Automated daily backup with cron

Terminal — crontab

```
$ crontab -e

# Add these lines:
# Database backup at 2:00 AM daily
0 2 * * * cd /home/yourname/uavlogbook_v3 && make backup >> /var/log/uavlogbook-backup.log 2>&1

# Delete backups older than 30 days
30 2 * * * find /home/yourname/uavlogbook_v3/backups -mtime +30 -delete
```

16 Troubleshooting

docker: command not found

Docker was not installed correctly, or you need to start a new terminal session. Re-run the install script: `curl -fsSL https://get.docker.com | sh`

```
$ curl -fsSL https://get.docker.com | sh
$ docker --version
```

permission denied connecting to Docker socket

Your user is not in the docker group, or the group change hasn't taken effect.

```
$ sudo usermod -aG docker $USER
$ newgrp docker
# If still failing, log out completely and log back in
$ docker ps    # should work now without sudo
```

docker compose: unknown command

Compose v2 plugin is missing. Install it:

```
$ sudo apt-get install -y docker-compose-plugin
$ docker compose version
```

Port 8080 already in use

Another process is using port 8080.

```
# Find what is using it:
$ sudo lsof -i :8080
$ sudo ss -tlnp | grep 8080

# Change the port in .env:
APP_PORT=9090
$ make down && make up
```

App container exits or keeps restarting

Check logs — most commonly a wrong password in .env or MySQL not yet ready.

```
$ docker compose logs app | tail -30
$ docker compose logs db | tail -30

# Common fixes:
# 1. .env missing: cp .env.example .env && nano .env
# 2. Password mismatch: ensure MYSQL_PASSWORD matches in .env
# 3. MySQL still starting: wait 60s, it imports schema on first run
```

Blank page / cannot reach the app from another machine

Firewall may be blocking the port, or you are using 'localhost' from another machine.

```
# Find the Ubuntu machine's IP:
$ ip addr show | grep 'inet ' | grep -v 127.0.0.1

# Open the firewall:
$ sudo ufw allow 8080/tcp
$ sudo ufw reload

# Access from another machine using the IP:
http://192.168.1.50:8080 ← not localhost
```

MySQL health check never passes / app waits forever

On first run MySQL imports the schema — this takes 60–90 seconds on slow hardware.

```
# Watch the database logs:
$ docker compose logs -f db

# Wait for this line:
# ready for connections

# If MySQL is stuck, reset and start fresh:
$ docker compose down
$ docker volume rm uavlogbook_db_data
$ docker compose up -d
```

'No space left on device' error

Disk is full. Docker images and volumes need space.

```
# Check disk usage:
$ df -h /
$ docker system df

# Clean up unused Docker resources:
$ docker image prune -a -f
$ docker system prune -f

# Check what is using disk:
$ du -sh ~/uavlogbook_v3/backups/*
```